



Python Course 2025/26/2

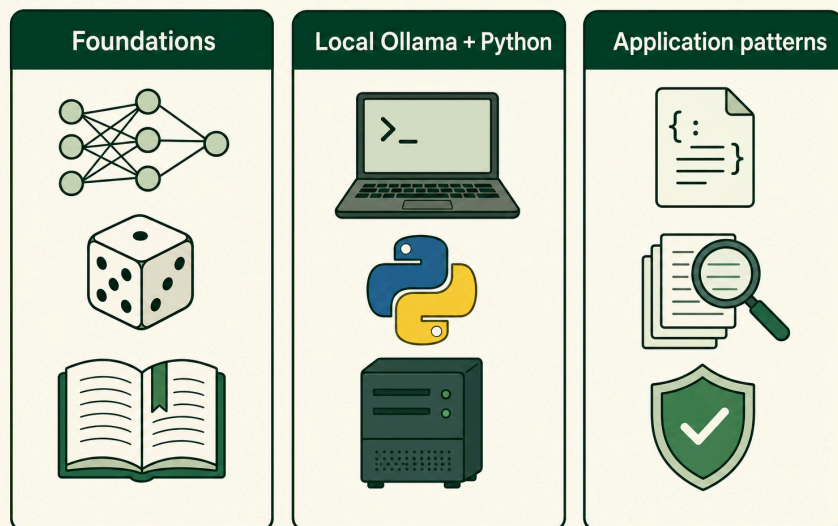
Lecture 12: Applied Python for Generative AI & Local Inference



Course coordinator: Dr Mate Tejfel

Lecturer: Zoltan Kegli

Lecture Overview



One story in three layers: ideas, then a working local call path, then patterns for real programs.

Learning objective

Connect the Lecture 9–11 to **generative** models you run locally: mechanism, Python call path, evaluation, retrieval, and safety.

By the end, you can

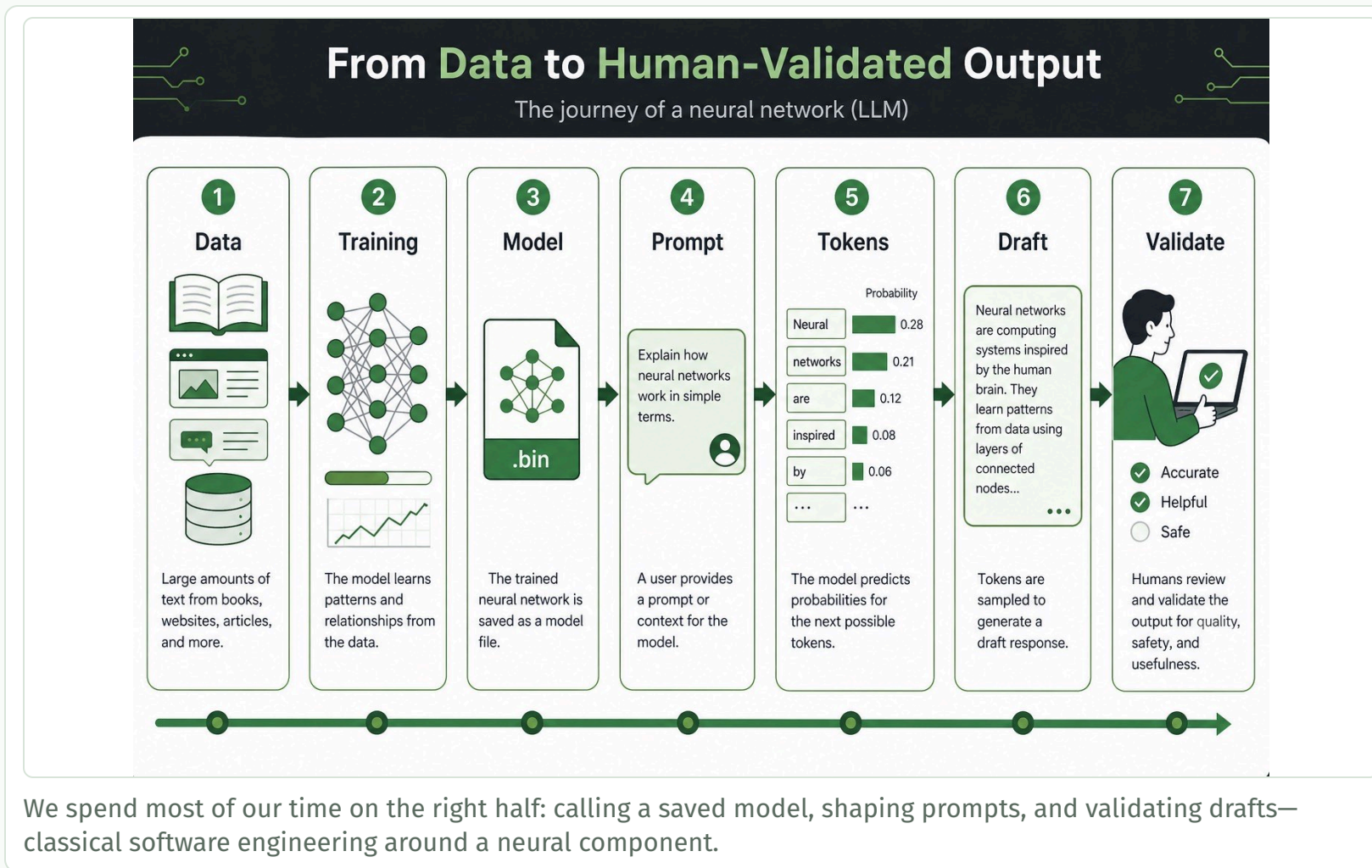
- **Foundations** — explain tokens, logits, attention, sampling, context windows, and quantization.
- **Local stack** — run `ollama.chat`, inspect drafts, store message history, and stream tokens.
- **Applications** — validate JSON with Pydantic, evaluate drafts, use RAG/embeddings, and gate tools with guardrails.



Foundations of generative AI

Vocabulary, next-token prediction, attention intuition, sampling, and the hardware bridge before we run
Ollama

Roadmap: training vs inference today





From Classification to Generation

Five words to carry over from Lecture 11

- **tensor** — an n-dimensional array of numbers (scalar → vector → matrix → tensor).
- **forward pass** — run the network once: input tensor → output tensor, no weight updates.
- **logit** — a raw unnormalized score the network emits (can be any real number).
- **softmax** — turns a vector of logits into probabilities that sum to 1.
- **argmax** — returns the index of the largest entry in a vector.

Classifier (Lecture 11)

- Input: features (pixels, measurements, embeddings of an image) packed into a tensor.
- One forward pass → logits, **one per class** (e.g. 10 for MNIST digits).
- Apply softmax to read probabilities, or take **argmax** for the best class.

Generative text model (today)

- Input: the **tokens** seen so far (the running context) as an integer tensor.
- One forward pass → logits, **one per token in the vocabulary** (typically 30k–200k entries).
- Apply softmax, pick one next token (sample or argmax), append to the context, repeat.

New idea first: a **vocabulary** is the full list of tokens the model can emit. A **token** is one item from that list (often a subword, not always a whole word).

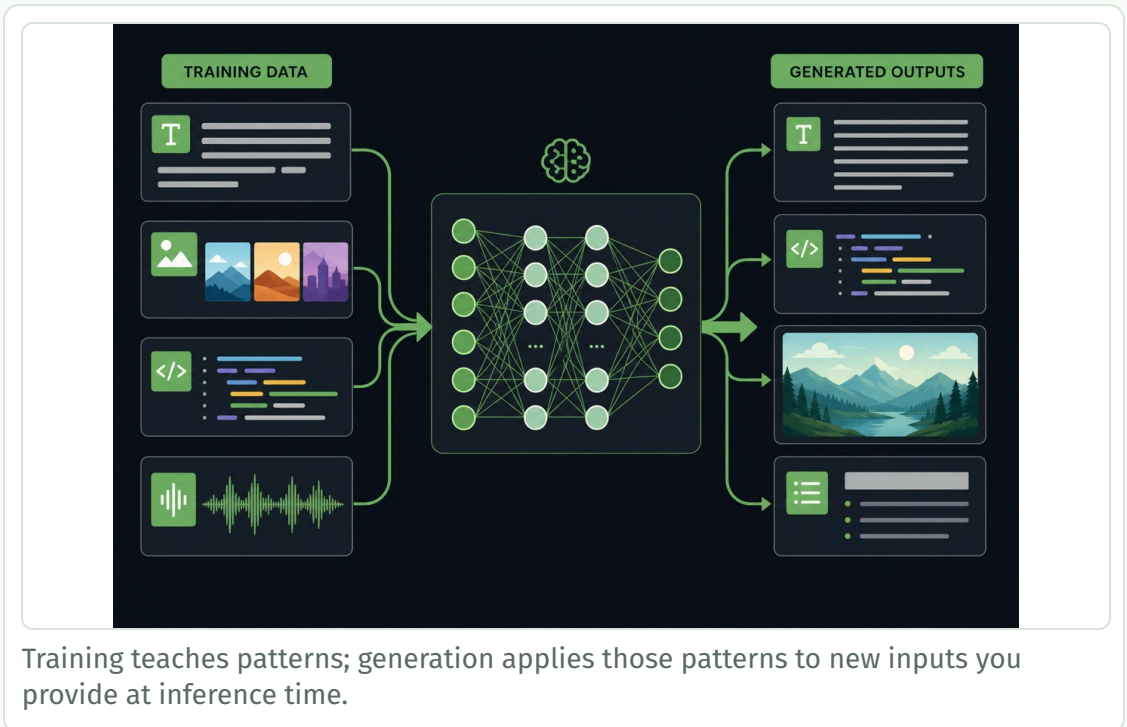


Vocabulary Words: AI, Model, Generative AI

Definitions in teaching order

- **Artificial intelligence (AI):** software that automates tasks we associate with human judgment (pattern recognition, language, planning).
- **Model:** the saved result of training: weights (the learned numbers from Lecture 11) + architecture + tokenizer rules + templates.
- **Generative AI:** systems that **produce new examples** (text, code, images, audio, video), not only pick a label.

A **modality** is a kind of data (text, image, audio). A **multimodal** model accepts more than one modality in the same interface.





Foundation Models and the Training Loop

1. Collect data

Large text/code corpora plus other modalities when training multimodal systems.

2. Hide the next token

Self-supervised objective (labels come from the data itself — no humans label each example): hide a token, predict it from the context. Wrong answer → nudge the weights.

3. Ship a checkpoint

A **foundation model** is a broad base many products fine-tune, prompt, or retrieve against.

Link to Lecture 11

Same ingredients — forward pass, loss, optimizer — but the target is a full probability vector over the vocabulary, not one softmax row over classes.

Tokenization: Text Becomes Integer IDs

What tokenization is

Tokenization is a deterministic program that maps text to a list of **token IDs** (integers indexing the vocabulary). It runs before any neural math.

Text

"unbelievable"

Pieces

un

believ

able

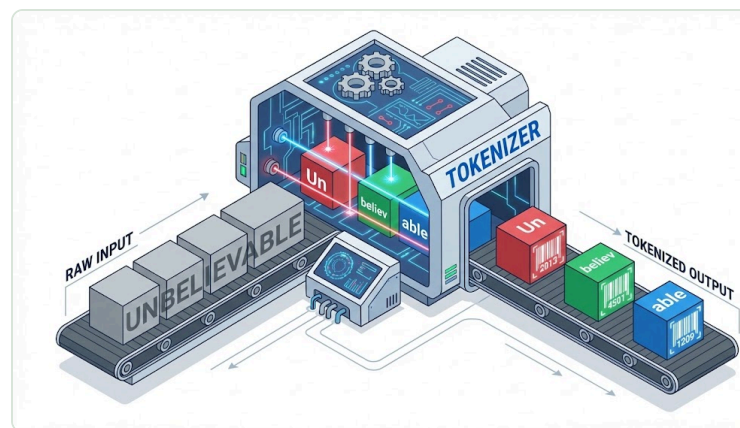
IDs

[403, 18124,
481]
(illustrative)

Tensor

`torch.LongTensor`
in PyTorch

Tensor recall: an n-dimensional array. A single prompt becomes a 1-D integer tensor of shape `(seq_len,)`; batched requests add a leading dimension → `(batch, seq_len)`. Same data type (integers) as a lookup table index.



Different models ship different tokenizers. Same Python string → different token counts → different latency and cost.

Logits, Softmax, One Probability Vector

From scores to probabilities

- **Logits:** the network's raw unnormalized scores, one per vocabulary entry. Can be negative, any magnitude.
- **Softmax:** exp-then-normalize (logits $\mathbf{x}$ are one score per vocabulary entry).

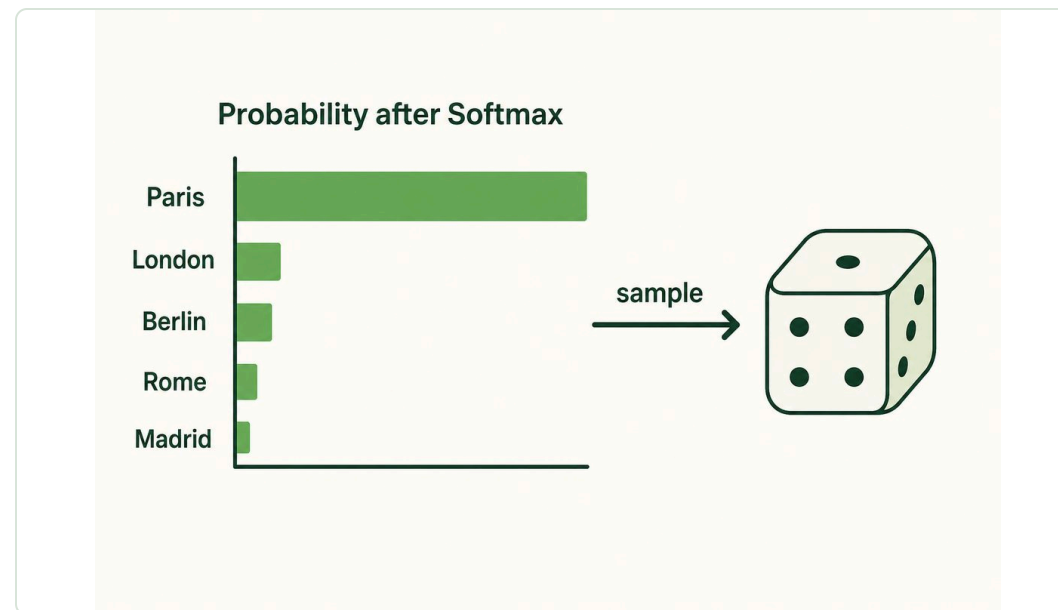
$$\text{softmax}(\mathbf{x})_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

`exp` makes every entry positive; dividing by the sum makes them add up to 1 → a valid probability distribution.

- **Argmax:** “arg max” = argument of the max. Returns the index of the largest entry — the greedy pick.
- **Autoregressive:** auto (self) + regressive (predicting from past values). Each new token is conditioned on every token chosen so far, including the ones the model just emitted.

Softmax is bookkeeping, not truth. A peaked distribution can still be confidently wrong.

Illustrative next-token mass



Toy view: real models expose tens of thousands of logits; demos plot only the top few for readability.

Generation Loop and Prompt

Prompt + loop

A **prompt** is the user or developer text (plus system instructions) that seeds the context.

1. Tokenize the prompt.
2. Forward pass → probabilities for the next token.
3. Sample or take argmax.
4. Append token; stop when a stop token or length budget hits.

```
context = tokenize("Write a haiku about Python")

while not done(context):
    logits = model.forward(context)
    probs = softmax(logits)
    next_id = sample(probs) # or argmax(probs): always pick the top (greedy)
    context.append(next_id)
```

Pseudocode for intuition. Production stacks add KV-cache, batching, streaming, and safety filters.

Why errors compound

- Each chosen token becomes part of the next context.
- An early mistake steers future logits even if later steps look confident.
- This is why retrieval, tools, tests, and human review matter.

Stop token: a reserved token or API flag meaning "finished".



Transformer Intuition: Attention and Cache

Attention

For each next token, the model compares the current position with earlier tokens and decides which parts of the context are useful. It is learned lookup-and-mixing over vectors, not symbolic memory.

Context engineering

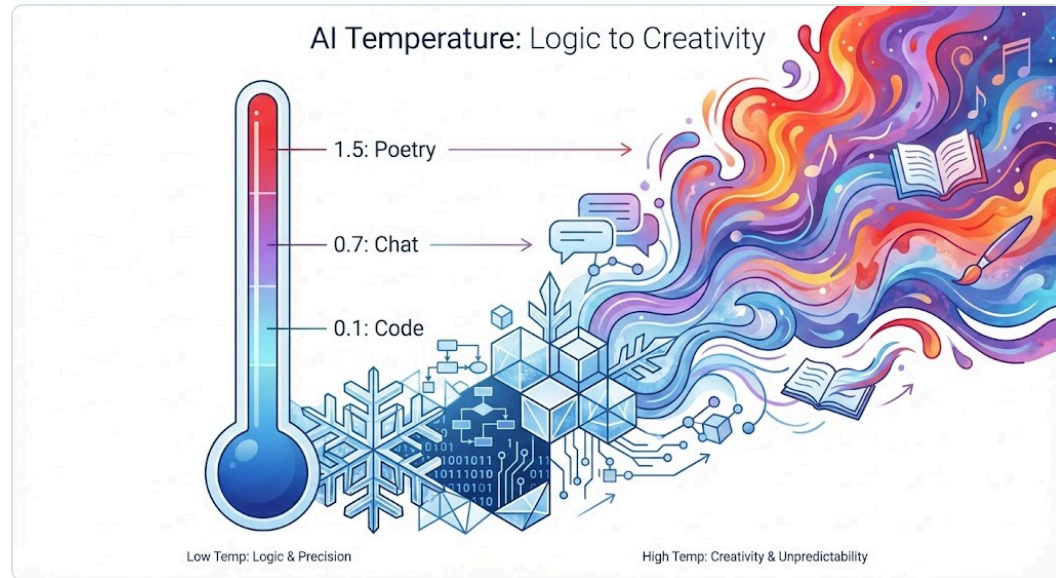
The model can only attend to tokens inside the request. Your Python code decides what history, retrieved chunks, examples, and instructions fit into that limited window.

KV cache

Runtimes cache internal key/value vectors for previous tokens, so generation can reuse prior work. Long prompts still cost time during **prefill**.

Good prompts are not magic wording. They are careful context construction: the right task, facts, examples, constraints, and output contract in the smallest useful token budget.

Sampling Knobs: Temperature, Top-k, Top-p



After softmax you may take the argmax (greedy) or draw a random sample proportional to probability.

Three controls

- **Temperature T:** divide logits by T before softmax → $\text{softmax}(\text{logits} / T)$. Low T sharpens the distribution ($T \rightarrow 0$ is basically argmax); high T flattens it, so less likely tokens stand a chance.
- **Top-k:** keep only the k highest-probability tokens, renormalize, sample. Simple, ignores the long tail.
- **Top-p (nucleus):** keep the smallest set whose cumulative mass $\geq p$, then sample. Adapts to the shape of the distribution — wider when the model is uncertain, narrower when it is confident.

Task	Typical randomness
JSON / extraction	greedy or very low temperature
Code assist	low to medium + top-p
Brainstorming	medium + nucleus sampling



Modalities and Where Python Sits

What can be generated?

Modality	Example outputs
Text	answers, summaries, translations
Code	functions, tests, refactors
Image / audio / video	edits, narration, storyboards

Python is the control plane

Models propose tokens; your code chooses models, trims context, calls tools, enforces permissions, logs prompts, and validates outputs with pytest or data checks.

Multimodal inputs (screenshots, PDFs) explode token counts—budget before you attach images to every request.



One Mental Model: Represent, Predict, Control

Represent

Tokenize text, normalize tables, resize images—turn raw inputs into tensors the network can consume.

Predict

Run the trained deep network to map context to logits, apply softmax, then sample or take the argmax.

Control

Prompt design, retrieval, structured outputs, monitoring, least-privilege tools, humans in the loop.

Generative AI is applied data science: data hygiene, modeling, evaluation, and software engineering in one loop.

Bridge to the Local Stack: Memory, Quantization, Models

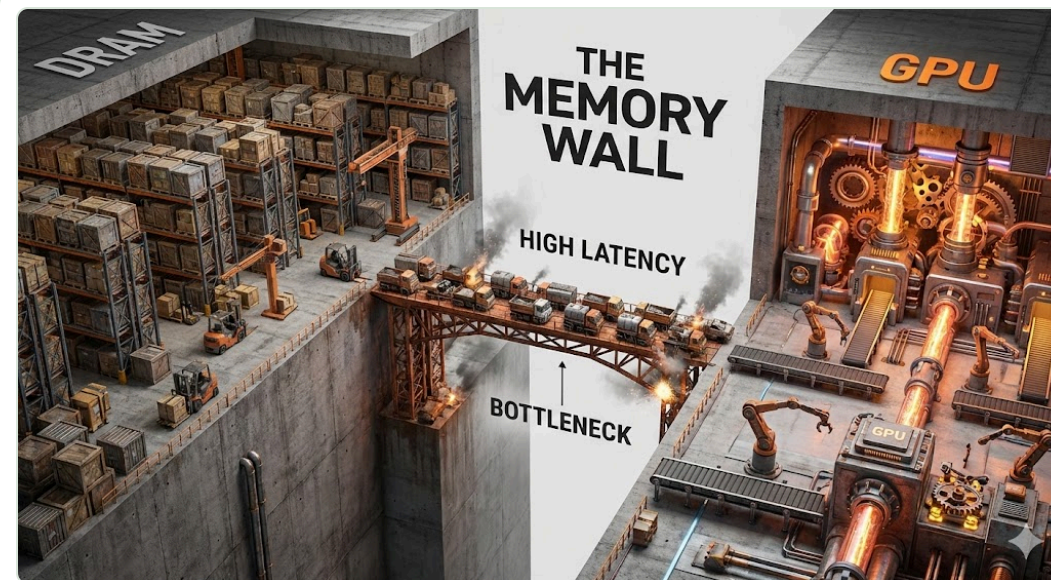
Why laptops care about bandwidth

`tokens_per_sec` $\approx$ `memory_bandwidth` $\div$

`bytes_per_weight_pass`

- Each new token touches every weight once in a naive matmul loop.
- Quantization packs more weights per gigabyte transferred $\rightarrow$ higher token throughput.

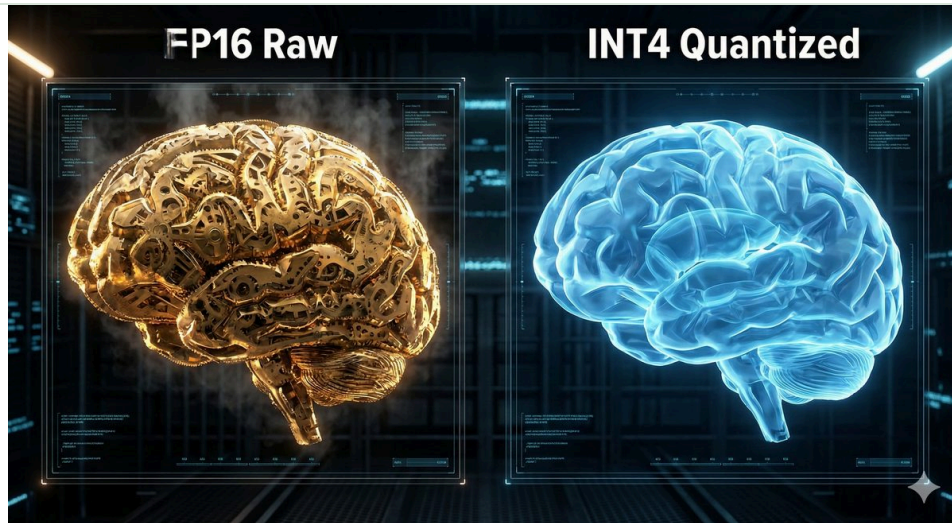
Section 2 shows the same tags running under Ollama from Python (`gemma4:e4b`, `embeddinggemma`).



If weights cannot reach the ALU fast enough, FLOPs are irrelevant.

Use case	Model tag	Approx size	Note
Validated lecture default	<code>gemma4:e4b</code>	9.6 GB	Pair with <code>embeddinggemma</code> for local RAG demos
Small text compare	<code>qwen3:4b</code>	2.5 GB	Great for experiments; chat templates may be noisy on some builds
Lab workstation	<code>qwen3:30b</code>	19 GB	Strong reasoning when VRAM/RAM allows

Quantization, GGUF, and Perplexity (PPL)



Quantization stores weights with fewer bits per number. Same chip, smaller payloads across the memory bus.

GGUF in one sentence

A single on-disk bundle holds quantized weights, lookup tables, tokenizer tables, and chat templates. Runtimes can **memory-map** (`mmap`) slices so cold pages never touch RAM (GPT-Generated Unified Format).

Perplexity (PPL)

$$\text{PPL} = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log P(x_i^* \mid x_{<i}) \right)$$

- Measures average surprise on real text; **lower is better**.
- Useful for comparing quant formats; pair with task evals (coding, retrieval) before shipping.



Checkpoint: Foundations

Q1

What makes an AI system “generative”?

- A) It only chooses one label from a fixed list
- B) It creates a new draft such as text, code, image, or audio
- C) It stores every training example in a database table
- D) It never needs human review

Show Answer

Q2

In an autoregressive language model, what happens after one next token is selected?

- A) The token is appended to the context and influences the next prediction
- B) The model changes its weights permanently
- C) The tokenizer is discarded for the rest of the answer
- D) The model switches from probabilities to a database lookup

Show Answer



Checkpoint: Foundations (continued)

Q3

Why does a long prompt often increase time-to-first-token even when generation is streamed?

- A) The model must prefill/cache the prompt tokens before emitting the first new token
- B) Streaming retrains the tokenizer before every answer
- C) The context window becomes unlimited
- D) Top-p sampling disables the forward pass

Show Answer

Q4

Why does weight quantization usually increase tokens per second on the same laptop?

- A) Fewer bits per weight means the memory subsystem delivers more useful parameters per second to the math units
- B) Quantization trains the model again from scratch
- C) It disables the tokenizer to save time
- D) It replaces Python with assembly everywhere

Show Answer



Local stack: Ollama from Python

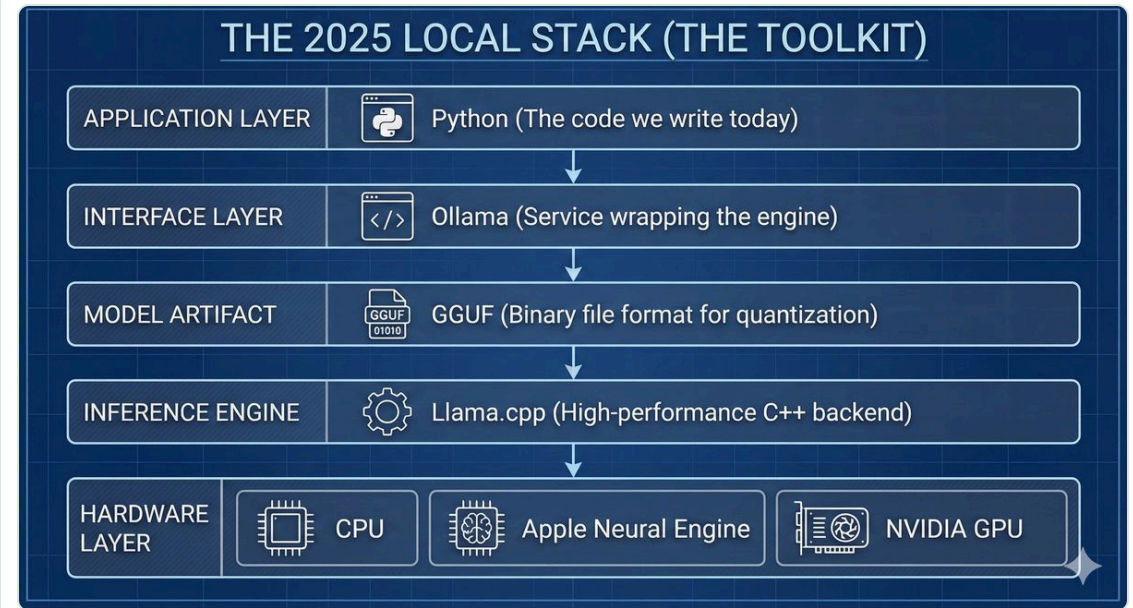
First local call, useful drafts, conversation state in your program, streaming, and time-to-first-token

What Ollama is (before any code)

Definition

Ollama is a program that runs on your computer, downloads **open-weight** models (weights you can download — license still matters), and serves them over local HTTP. The `ollama` Python package is a small client that sends requests to that server.

- You already saw **GGUF** in the foundations section: one file bundles quantized weights plus tokenizer metadata Ollama needs.
- **Inference** here means “ask the loaded model for the next tokens”; no training happens in these demos.



Python orchestrates prompts and business logic; Ollama concentrates on efficient token generation.

First call: `ollama.chat`

```
import ollama

response = ollama.chat(
    model="gemma4:e4b",
    messages=[
        {"role": "system", "content": "You are a careful Python tutor."},
        {"role": "user", "content": "Define a token in one short sentence."},
    ],
    options={"temperature": 0.3, "num_ctx": 4096},
)

print(response.message.content)
```

Aligned with `src/02_ollama_basics_and_state.py`: same defaults and metric helpers live in `demo_utils`.

Read the response object

- **Role:** each message is tagged `system`, `user`, or `assistant` so the runtime can apply the model's chat template.
- **Model name string:** the tag must match a model you pulled with `ollama pull` (for example `gemma4:e4b`).
- **options:** classroom knobs include `temperature` and `num_ctx` (how many tokens of context the runtime may use).



Strength Demo: Natural Language → Python Draft

Prompt

"Write a Python function that groups transactions by month."

Useful first draft

```
from collections import defaultdict

def group_by_month(rows):
    groups = defaultdict(list)
    for row in rows:
        groups[row["date"][:7]].append(row)
    return dict(groups)
```

Why this belongs after the first call

- Students have now seen the exact Python API that produces the draft.
- The output is useful because a draft saves time, not because it is automatically correct.
- Follow-up prompts can explain, refactor, or adapt it to pandas, SQL, or tests.

Engineering habit

Always pair generation with checks: empty input, malformed dates, time zones, huge files.

Statelessness: memory lives in Python



```
messages = [
    {"role": "system", "content": "You are a careful tutor."},
    {"role": "user", "content": "My name is Bela."},
    {"role": "assistant", "content": "Nice to meet you, Bela."},
    {"role": "user", "content": "What is my name?"},
]

ollama.chat(model="gemma4:e4b", messages=messages)
```

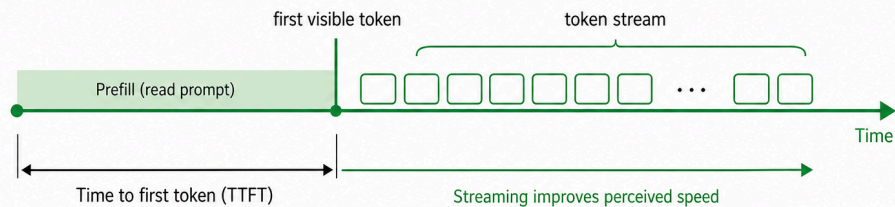
If you omit earlier turns, the model cannot see them — there is no hidden notebook inside the weights for your chat.

Terms in the list literal

- **System message:** developer-facing instructions (style, safety, output format).
- **User / assistant turns:** mimic a conversation transcript.
- **Conversation state:** the growing `messages` list you store, trim, summarize, or augment — classic Python data structures, not magic model memory.

Demo 2 in the notebook proves forgetting: change the list, watch the answer change.

Streaming tokens and time-to-first-token (TTFT)



Streaming forwards partial output while generation continues; users see progress sooner.

Define the metrics

- **Time to first token (TTFT):** seconds from sending the request until the first piece of generated text arrives.
- **Prefill:** the work the runtime does to ingest your prompt (and prior context) before it can emit the first newly generated token; large prefill often dominates TTFT.
- **Throughput:** tokens per second after streaming starts — still important, but TTFT drives perceived snappiness in chat UIs.

```
stream = ollama.chat(  
    model="gemma4:e4b",  
    messages=[{"role": "user", "content": "Stream a haiku about pandas."}],  
    stream=True,  
)  
  
for chunk in stream:  
    if piece := chunk.message.content:  
        print(piece, end="", flush=True)
```

Async AsyncClient and FastAPI StreamingResponse build the same idea for web APIs; see `src/03_streaming_and_fastapi.py`.



Validated demo defaults (your machine may vary)

Always match code, slides, and the README: students should **pull** the same tags before class.

Role	Model tag	Notes
Chat default	gemma4 : e4b	Pairs with lecture checkpoints; balanced quality on modest GPUs/CPUs.
Embeddings for RAG demos	embeddinggemma	Vectors for semantic search examples in section 3.
Small alternative	qwen3 : 4b	Great for experiments; chat templates differ by release.



Checkpoint: Local Ollama stack

Q5

Where does multi-turn chat memory live unless you code it yourself?

- A) Inside the model weights forever
- B) In a hidden Ollama database that remembers every user
- C) In the `messages` list your Python program passes to each call
- D) Only in the cloud provider's logs

Show Answer

Q6

Why do UIs enable streaming instead of always waiting for the entire answer string?

- A) Streaming guarantees factual correctness
- B) Streaming lowers TTFT so users see the first tokens earlier
- C) Streaming removes the need for a tokenizer
- D) Streaming trains the model weights during the request

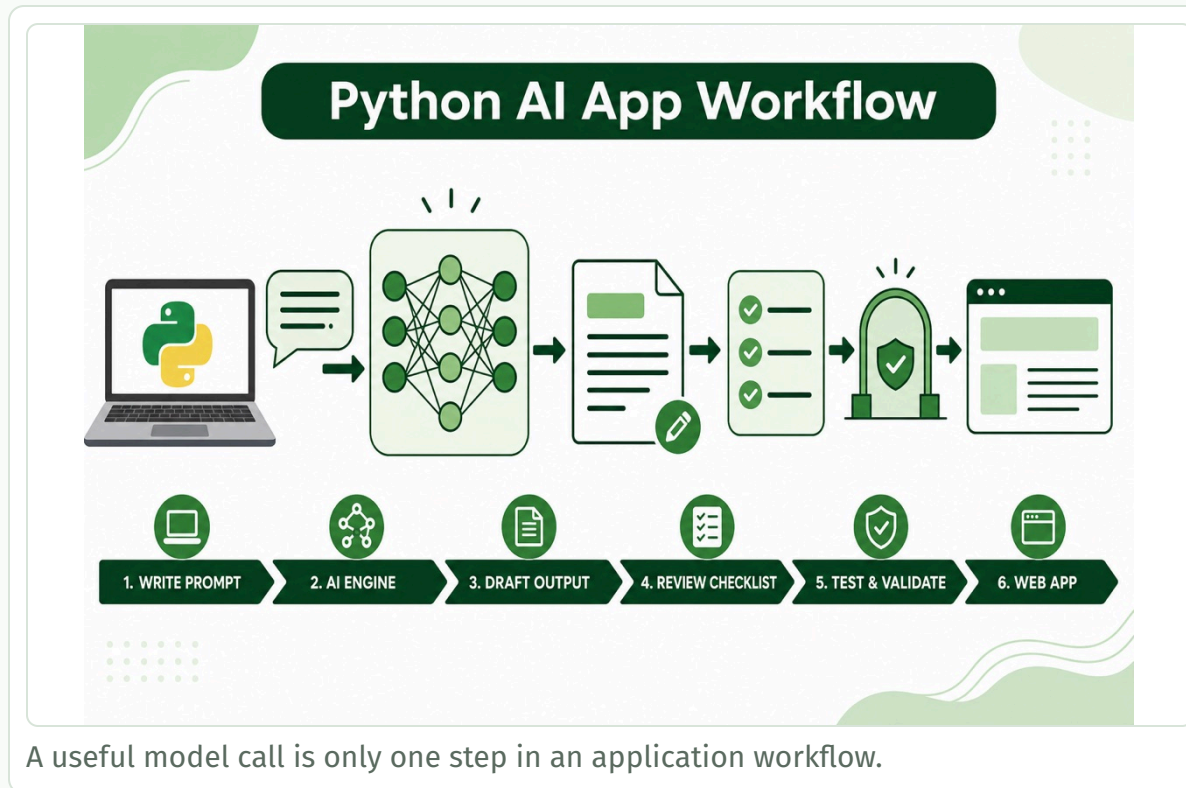
Show Answer



Application patterns

Use cases, evaluation, structured outputs, grounding, embeddings, RAG, tools, agents, guardrails

The Python App Workflow: Ask, Draft, Check



Define the words before using them

- **Prompt:** the instruction and input text sent to the model.
- **Context:** everything the model sees in one call: prompt, history, documents, and system instructions.
- **Inference:** running the already-trained model to produce output; no weights are changed.
- **Draft:** a candidate answer produced by the model, not a verified fact or finished program.

Core habit: generative AI is strongest when a checked draft is useful. Python should still validate, test, and decide what is allowed.



Where GenAI Helps — and Where It Fails

Strong fits

- Draft code, tests, docs, emails, and explanations.
- Transform text between formats when mistakes are cheap to catch.
- Search, summarize, and tutor over trusted material.

Model risks

- **Hallucination:** fluent but false or unsupported claims.
- **Bias:** skewed outputs from data, prompts, or evaluation gaps.
- **Stale knowledge:** APIs and facts move faster than weights.

System risks

- Latency spikes, token budgets, and denial-of-wallet failures.
- Privacy leaks through prompts, logs, or generated HTML.
- Non-determinism when sampling is on, even with the same prompt.

Ship only where a checked draft is useful. If the workflow cannot detect a bad answer, the model is in the wrong place.



Evaluation: Make Quality Measurable

CS habit: tests before trust

- **Golden set:** small representative inputs with expected behavior.
- **Unit checks:** schema validity, no missing fields, no dangerous sinks.
- **Regression run:** rerun the same cases after model, prompt, or retrieval changes.
- **Human review:** sample hard cases where correctness is semantic.

Measure by task

Task	Useful checks
Extraction	JSON schema + field accuracy
Code draft	pytest + review checklist
RAG answer	context relevance, groundedness, answer relevance
Tool call	authorization + dry-run + audit log

Modern GenAI engineering is less "find the perfect prompt" and more "keep a repeatable evaluation harness beside every prompt, schema, retriever, and tool."



Structured Output: Define the Shape Before You Ask

```
from pydantic import BaseModel, Field
from ollama import chat

class StudyCard(BaseModel):
    term: str = Field(..., min_length=1)
    definition: str = Field(..., min_length=20)
    python_example: str = Field(..., min_length=10)

response = chat(
    model="gemma4:e4b",
    messages=[{"role": "user", "content": "Create a study card for 'token'."}],
    format=StudyCard.model_json_schema(),
    options={"temperature": 0},
)

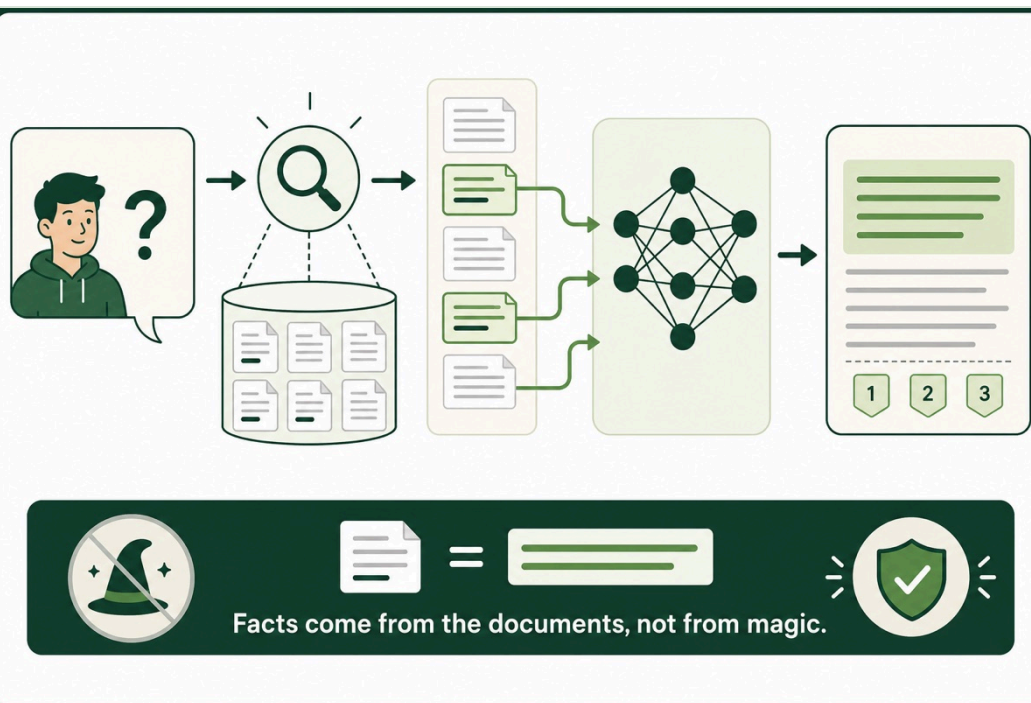
card = StudyCard.model_validate_json(response.message.content)
print(card.term, "⇒", card.definition)
```

Definitions

- **JSON:** a text format for structured data: objects, lists, strings, numbers, booleans.
- **Schema:** a formal description of allowed fields, types, and constraints.
- **Pydantic model:** a Python class that validates data against such a shape.

Structured output makes model output easier to connect to Python functions, databases, and APIs. It reduces formatting mistakes, but validation is still required. For best reliability, pass the JSON schema to `format`, set a low temperature, and ask for JSON in the prompt too.

Grounding: Give the Model the Facts



Facts should come from selected documents, not from a lucky guess.

Build the idea step by step

- **Grounding:** attaching the answer to provided facts.
- **Retrieval:** searching a knowledge source before asking the model to answer.
- **Chunk:** a small piece of a document that can fit into context.
- **Citation:** an identifier showing which chunk supported the answer.
- **RAG:** Retrieval Augmented Generation: retrieve facts, add them to the prompt, then generate.

Use RAG when the answer depends on private, changing, or course-specific documents.



Embeddings: Searching by Meaning

```
import math
import ollama

docs = [
    {"id": "returns", "text": "Opened items can be returned with a fee."},
    {"id": "shipping", "text": "Standard delivery takes 3 to 5 days."},
]

def embed(texts):
    return ollama.embed(model="embeddinggemma", input=texts)["embeddings"]

def dot(a, b):
    return math.fsum(x * y for x, y in zip(a, b))

doc_vectors = embed([doc["text"] for doc in docs])
query_vector = embed("Can I get my money back after opening the box?")[0]

best_score, best_doc = max(
    (dot(query_vector, vec), doc)
    for doc, vec in zip(docs, doc_vectors)
)
print(best_doc["id"], round(best_score, 3))
```

Definitions

- **Embedding:** a list of numbers representing the meaning of a text for search.
- **Vector:** the math name for that list of numbers.
- **Similarity score:** a number saying how close two vectors are.
- **Cosine similarity:** a common score based on vector direction; Ollama embeddings are normalized, so dot product works as the score.

This lets "money back" match "return policy" even when the words are not identical.



RAG Quality: Retrieve, Ground, Answer

1. Retrieval quality

Did search fetch the right chunks? Improve with better chunking, metadata filters, hybrid keyword + vector search, or reranking.

2. Groundedness

Does every important claim come from the retrieved context? Ask for citations and reject unsupported claims in evaluation.

3. Answer quality

Does the response answer the actual question in the right tone and format? This is measured against user-visible examples.

Security-specific checks

- Filter retrieval by user permissions before the model sees chunks.
- Treat retrieved text as data, not instructions.
- Block or quarantine obvious prompt-injection snippets.

2026 default: evaluate the retrieval step separately from the answer step. A better model cannot fix missing, unauthorized, or poisoned context.



RAG vs Fine-Tuning: Which Lever Changed?

Fine-tuning means continuing training so the model weights change. **RAG** means leaving weights unchanged and changing the context supplied at inference time.

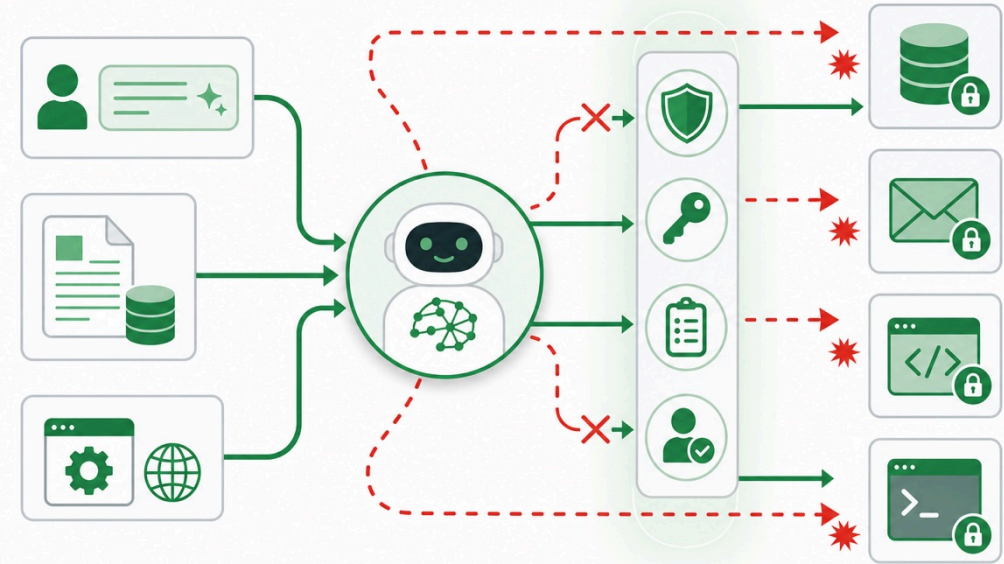
Question	Prefer RAG	Prefer fine-tuning
Need fresh facts?	Yes: update the document index.	No: weights go stale after training.
Need private course or company data?	Yes: keep access checks in the database or search layer.	Risky: data is baked into weights and hard to revoke.
Need a stable style or behavior?	Sometimes: prompt templates may be enough.	Often: examples can teach a repeated response habit.
Cost for a first prototype?	Lower: retrieval + normal model calls.	Higher: training data, GPUs, and evaluation.

Tools, Agents, and Guardrails

Definitions first

- **Tool:** a normal function or API the application can call, such as `search_docs()` or `send_email()`.
- **Agent:** a program loop where the model proposes the next step and the application decides whether to run it.
- **Guardrail:** an external check that blocks unsafe or invalid actions.
- **Authorization:** a permission check: is this user allowed to do this?

The model may suggest an action; Python enforces permissions, input validation, budgets, dry-runs, approval gates, and logging.



Security boundaries live outside the model response.

2026 Direction: Narrow, Evaluated, Governed



The mature pattern is a scoped workflow with metrics, not a chatbot that can do everything.

Proved production direction

- Small or specialized models when they meet the task; reserve huge models for hard cases.
- Structured outputs and tool calls with validation instead of free-text glue code.
- RAG with access control, citations, and retrieval evaluation for changing/private facts.

Agentic caution

- Plan-and-act loops need step-level evaluation, budget limits, and cancellation paths.
- Human approval remains the default for irreversible or high-impact actions.
- System prompts are guidance, not secrets and not authorization.



Security Checklist for Python GenAI Apps

Inputs

- **Prompt injection:** direct user text or indirect text inside retrieved documents tries to override instructions.
- Treat uploaded documents, web pages, chat messages, and tool outputs as untrusted data.
- Never place API keys, credentials, or private policy in a system prompt.

Actions

- Run authorization checks before every tool call.
- Use allow-lists for files, URLs, commands, models, and database operations.
- Set token, cost, and loop limits; ask for human approval before irreversible actions.

Outputs

- Validate JSON with Pydantic before using it.
- Escape generated text before displaying it in HTML.
- Require citations for grounded answers; reject unsupported claims in high-stakes flows.
- Log enough metadata to debug, but avoid storing private prompts unnecessarily.

Safe design assumption: the model is helpful but not trusted. Current OWASP-style risks include prompt injection, sensitive information disclosure, system prompt leakage, vector/embedding weaknesses, excessive agency, and unbounded consumption. Your Python application is the security boundary.



Putting It Together: A Responsible GenAI Loop

1. Ask

Write the prompt and define the task.

2. Retrieve

Add the relevant facts when facts matter.

3. Generate

Run inference and receive a draft.

4. Validate

Check schema, tests, permissions, and citations.

5. Use

Show, store, call a tool, or ask a human to approve.

Good classroom demo

- Small local model via Ollama.
- Short prompts students can read.
- Pydantic for structured answers.
- In-memory retrieval so the algorithm is visible.

Good production direction

- Persistent document index and access control.
- Automated evaluation on real examples.
- Monitoring for latency, cost, and failures.
- Clear policy for privacy and human review.



Checkpoint: Application patterns

Q7

Your model returns a plausible Python function. What should your application or workflow do before using it?

- A) Trust it because the model used many training examples
- B) Review it, run tests, and check edge cases before integration
- C) Increase temperature to make the code more confident
- D) Hide the generated code from the programmer

Show Answer

Q8

A course chatbot must answer from this semester's PDF handouts. Which design best matches that requirement?

- A) RAG: retrieve relevant chunks from the handouts, then generate with citations
- B) Ask the model to remember the files without sending them
- C) Put secret API keys into the prompt so it has more context
- D) Fine-tune first, even for a one-week prototype

Show Answer



Lecture Summary

1. Foundations

- Builds on Lecture 9–11: tensors, forward pass, softmax—but the head now scores every vocabulary token, not one row of class labels.
- Tokenization turns Python strings into integer IDs; attention uses the visible context; sampling turns probabilities into the next ID.
- Quantization and memory bandwidth explain why smaller weights often mean faster local inference.

2. Local stack (Ollama)

- `ollama.chat` is the synchronous client path; useful drafts still need tests and review.
- **Time to first token (TTFT)** measures wait until the first generated chunk arrives; streaming helps even when total time is unchanged.
- Validated tags: `gemma4:e4b` for chat, `embeddinggemma` for embedding vectors in RAG demos.

3. Application patterns

- **Evaluation:** keep golden examples, schema checks, regression runs, and human review beside the prompt.
- **Structured output + RAG:** validate JSON, retrieve permitted chunks, then generate grounded answers with citations.
- **Tools / agents / guardrails:** the model proposes; Python authorizes, budgets, validates, and logs.

A local **large language model (LLM)** behaves like a stateless service call: it sees only the messages and options you send now. Session memory, retrieval, JSON parsing, evaluation, and security are entirely your Python layer.



Thank You for Your Attention

"The model has knowledge but no memory. Our job as engineers is to give it a desk."